

번호판 글자 분리 디버깅 노트북

`split_by_projection` 함수의 단계별 중간 결과를 시각화하여 알고리즘 동작을 검증합니다.

처리 흐름

1. 번호판 이미지 로드 + 정렬 (rectification)
2. 그레이스케일 변환
3. Otsu 이진화
4. 상하 노이즈 제거 (10% 트리밍)
5. 세로 방향 픽셀 합 (projection profile)
6. 프로파일 스무딩
7. 골짜기(valley) 검출
8. 글자 경계 결정
9. 글자별 크롭
10. 글자 수 보정 (필요 시)

각 단계마다 시각화를 출력합니다.

```
In [1]: # 필요 라이브러리 임포트
import cv2
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rcParams
from scipy.signal import find_peaks
from pathlib import Path

# 한글 폰트 설정 (시스템에 맞게 변경)
# Windows: 'Malgun Gothic', macOS: 'AppleGothic', Linux: 'NanumGothic'
rcParams['font.family'] = 'Malgun Gothic'
rcParams['axes.unicode_minus'] = False
rcParams['figure.dpi'] = 100

print('환경 준비 완료')
```

환경 준비 완료

설정

테스트할 번호판 이미지 경로를 지정합니다. 정렬(rectified)된 번호판 이미지가 좋습니다
— 정렬되지 않은 원본을 쓰면 분리 결과가 부정확할 수 있습니다.

```
In [2]: # 번호판 이미지 경로 지정
PLATE_IMAGE_PATH = 'rectified_result.jpg' # ← 본인 이미지 경로로 변경
EXPECTED_CHARS = 7 # 7 또는 8 (None이면 자동 추정)

# 이미지 로드 (한글 경로 호환)
def load_image_safe(path):
```

```

img = cv2.imread(str(path), cv2.IMREAD_COLOR)
if img is None:
    try:
        buf = np.fromfile(str(path), dtype=np.uint8)
        img = cv2.imdecode(buf, cv2.IMREAD_COLOR)
    except Exception:
        return None
return img

if Path(PLATE_IMAGE_PATH).exists():
    rectified_plate = load_image_safe(PLATE_IMAGE_PATH)
    print(f'이미지 로드 성공: {rectified_plate.shape}')
else:
    # 테스트용 더미 번호판 생성 (실제 이미지가 없을 때)
    print('실제 이미지가 없어 테스트용 더미 이미지를 생성합니다.')
    rectified_plate = np.full((80, 280, 3), 240, dtype=np.uint8)
    cv2.putText(rectified_plate, '12 7 3456', (15, 60),
                cv2.FONT_HERSHEY_SIMPLEX, 1.6, (20, 20, 20), 3)

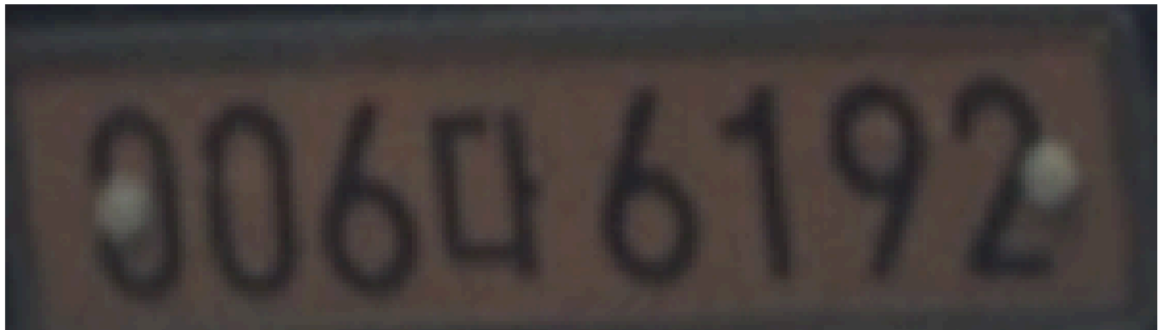
# 1단계: 원본 이미지 시각화
plt.figure(figsize=(10, 3))
plt.imshow(cv2.cvtColor(rectified_plate, cv2.COLOR_BGR2RGB))
plt.title('Step 1. 원본 (정렬된) 번호판 이미지')
plt.axis('off')
plt.tight_layout()
plt.show()

print(f'이미지 크기: {rectified_plate.shape[1]} x {rectified_plate.shape[0]}')

```

이미지 로드 성공: (80, 280, 3)

Step 1. 원본 (정렬된) 번호판 이미지



이미지 크기: 280 x 80

Step 2. 그레이스케일 변환

OCR에 색상 정보는 불필요하므로 단일 채널로 변환합니다.

```

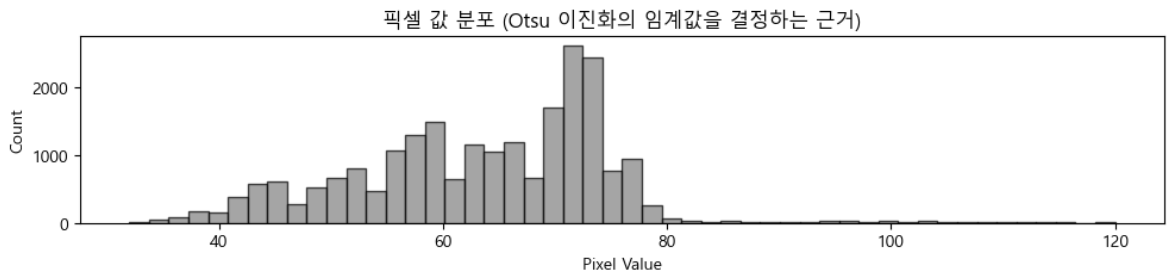
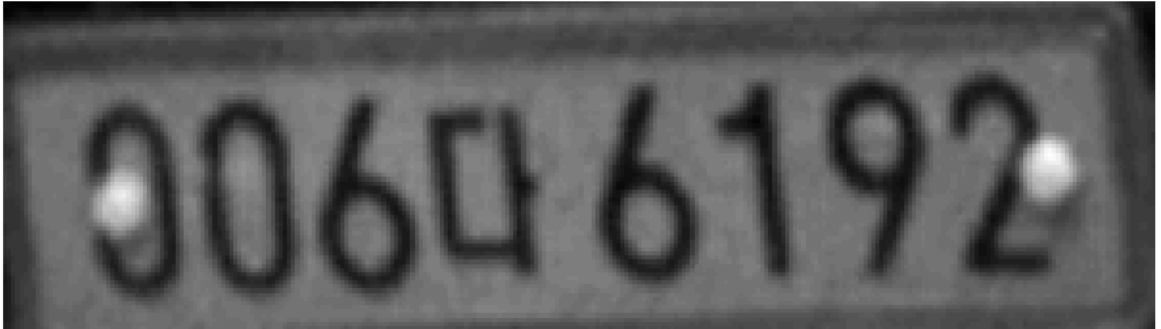
In [3]: if len(rectified_plate.shape) == 3:
        gray = cv2.cvtColor(rectified_plate, cv2.COLOR_BGR2GRAY)
    else:
        gray = rectified_plate.copy()

    plt.figure(figsize=(10, 3))
    plt.imshow(gray, cmap='gray')
    plt.title(f'Step 2. 그레이스케일 ({gray.shape[1]} x {gray.shape[0]})')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

```

```
# 픽셀 값 분포 히스토그램
plt.figure(figsize=(10, 2.5))
plt.hist(gray.ravel(), bins=50, color='gray', edgecolor='black', alpha=0.7)
plt.title('픽셀 값 분포 (Otsu 이진화의 임계값을 결정하는 근거)')
plt.xlabel('Pixel Value')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

Step 2. 그레이스케일 (280 x 80)



Step 3. Otsu 이진화

THRESH_BINARY_INV 로 글자=흰색(255), 배경=검정(0)이 되도록 반전합니다. 이 반전이 중요합니다 — 다음 단계의 픽셀 합이 글자 영역에서 커지도록 만들기 위함입니다.

만약 이미지에서 글자가 어둡고 배경이 밝다면 그대로 BINARY_INV 를 쓰면 됩니다. 글자가 밝고 배경이 어둡다면 BINARY (반전 없이)를 써야 합니다.

```
In [4]: # Otsu 이진화 (자동 임계값)
threshold_value, binary = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)
print(f'Otsu가 자동 결정한 임계값: {threshold_value}')

# 비교용으로 BINARY (반전 안 함)도 만들기
_, binary_no_inv = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

fig, axes = plt.subplots(1, 2, figsize=(14, 3))
axes[0].imshow(binary, cmap='gray')
axes[0].set_title(f'Step 3a. THRESH_BINARY_INV (글자=흰색)\n흰 픽셀 비율: {binary_no_inv.sum() / binary_no_inv.size}')
axes[0].axis('off')

axes[1].imshow(binary_no_inv, cmap='gray')
axes[1].set_title(f'Step 3b. THRESH_BINARY (글자=검정)\n흰 픽셀 비율: {binary.sum() / binary.size}')
axes[1].axis('off')
```

```
plt.tight_layout()
plt.show()

# 진단: 어느 쪽을 써야 하는지 자동 추천
if binary.mean() < binary_no_inv.mean():
    print('✓ BINARY_INV가 적절 (글자가 어두운 색)')
else:
    print('⚠ BINARY가 적절할 수도 있음 - 결과가 이상하면 반전을 바꿔보세요')
```

Otsu가 자동 결정한 임계값: 62.0



✓ BINARY_INV가 적절 (글자가 어두운 색)

Step 4. 상하 트리밍

번호판 위·아래 테두리나 약간의 노이즈를 제거하기 위해 상하 10%씩 잘라냅니다.

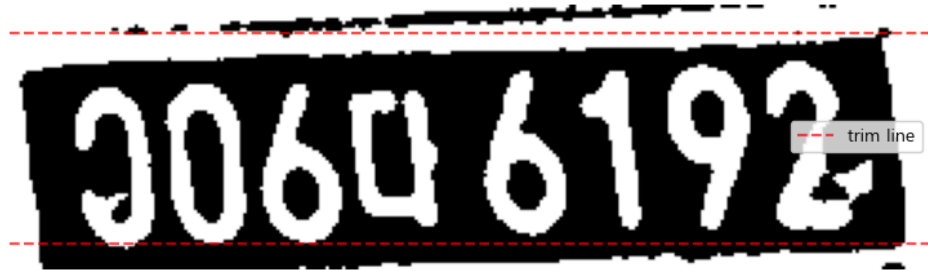
원본 코드는 `binary[int(h*0.1):int(h*0.9), :]` 로 상하 10%씩 총 20%를 잘라냅니다. 주석에는 5%로 적혀 있지만 실제 코드는 10%이므로 코드 기준으로 동작합니다.

```
In [5]: h = binary.shape[0]
trimmed = binary[int(h*0.1):int(h*0.9), :]

fig, axes = plt.subplots(2, 1, figsize=(12, 5))
axes[0].imshow(binary, cmap='gray')
# 트리밍 영역 표시
axes[0].axhline(int(h*0.1), color='red', linestyle='--', alpha=0.7, label='trim')
axes[0].axhline(int(h*0.9), color='red', linestyle='--', alpha=0.7)
axes[0].set_title(f'Step 4a. 트리밍 전 (h={h})')
axes[0].legend()
axes[0].axis('off')

axes[1].imshow(trimmed, cmap='gray')
axes[1].set_title(f'Step 4b. 상하 10% 트리밍 후 (h={trimmed.shape[0]})')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

Step 4a. 트리밍 전 (h=80)



Step 4b. 상하 10% 트리밍 후 (h=64)



Step 5. 세로 방향 픽셀 합 (Projection Profile)

각 가로 위치(x)에서 세로 방향으로 픽셀 값을 모두 더합니다. 결과는 1D 배열이며, 글자가 있는 위치는 값이 크고 글자 사이 공백은 값이 작아야 합니다.

이 프로파일의 골짜기(valley)가 곧 글자 사이 경계입니다.

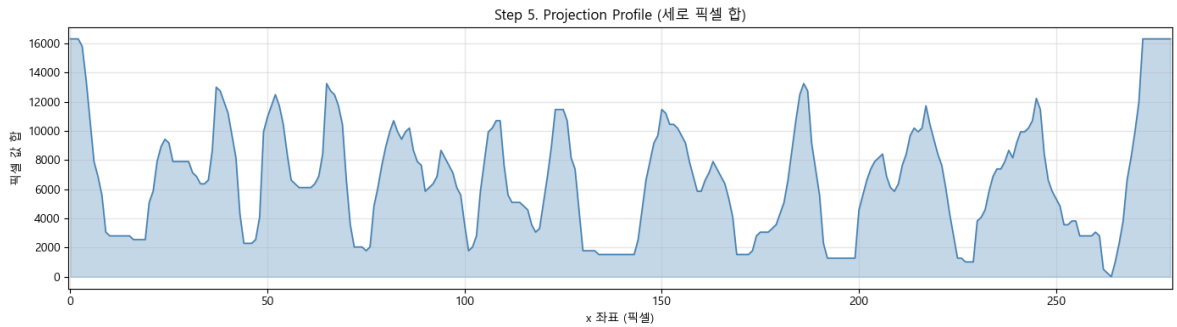
```
In [6]: # 세로 방향 합산
profile = np.sum(trimmed, axis=0)
print(f'프로필 길이: {len(profile)}')
print(f'최댓값: {profile.max()}, 평균: {profile.mean():.1f}, 최솟값: {profile.min():.1f}')

# 시각화: 이진화 이미지 + 프로필을 같은 x축으로
fig, axes = plt.subplots(2, 1, figsize=(14, 6),
                        gridspec_kw={'height_ratios': [1, 2]}, sharex=True)
axes[0].imshow(trimmed, cmap='gray', aspect='auto')
axes[0].set_title('이진화 이미지 (참고)')
axes[0].set_yticks([])

axes[1].plot(profile, color='steelblue', linewidth=1.2)
axes[1].fill_between(range(len(profile)), profile, alpha=0.3, color='steelblue')
axes[1].set_title('Step 5. Projection Profile (세로 픽셀 합)')
axes[1].set_xlabel('x 좌표 (픽셀)')
axes[1].set_ylabel('픽셀 값 합')
axes[1].grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

프로필 길이: 280

최댓값: 16320, 평균: 6681.0, 최솟값: 0



Step 6. 프로필 스무딩

원본 프로필은 픽셀 단위로 거칠어 작은 노이즈가 valley로 잡힐 수 있습니다. 이동 평균 (moving average)으로 부드럽게 만듭니다.

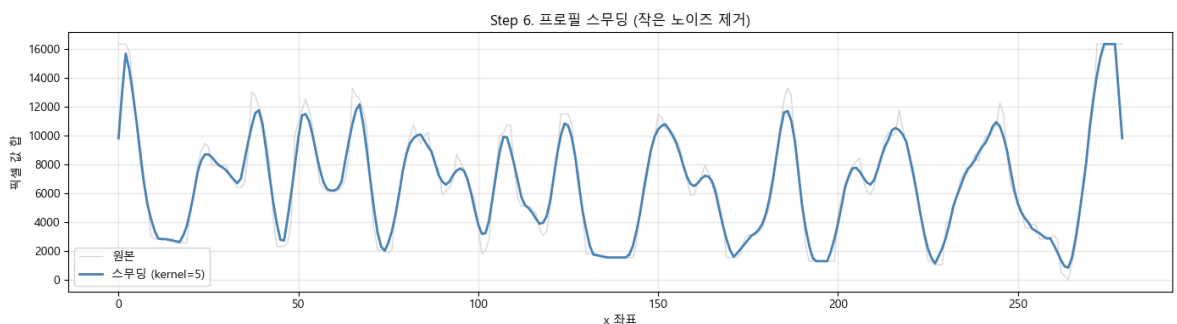
`kernel_size = max(3, len(profile) // 50)` 이므로 길이 280인 프로필은 커널 5 정도가 됩니다.

```
In [7]: kernel_size = max(3, len(profile) // 50)
smoothed = np.convolve(profile, np.ones(kernel_size) / kernel_size, mode='same')

print(f'스무딩 커널 크기: {kernel_size}')

plt.figure(figsize=(14, 4))
plt.plot(profile, color='lightgray', linewidth=1, label='원본', alpha=0.8)
plt.plot(smoothed, color='steelblue', linewidth=2, label=f'스무딩 (kernel={kernel_size})')
plt.title('Step 6. 프로필 스무딩 (작은 노이즈 제거)')
plt.xlabel('x 좌표')
plt.ylabel('픽셀 값 합')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

스무딩 커널 크기: 5



Step 7. Valley(골짜기) 검출

`scipy.signal.find_peaks` 는 봉우리를 찾는 함수이므로, 음수로 뒤집어서 골짜기를 찾습니다.

핵심 파라미터:

- `distance=len(profile) // 12` : valley 간 최소 거리. 7~8글자가 들어갈 만큼의 간격.
- `height=-threshold` : valley의 깊이 임계값 (`threshold = max * 0.3`)

이 임계값들이 분리 품질에 큰 영향을 줍니다. 이 셀에서 값을 조정하며 결과를 비교하세요.

```
In [8]: # 임계값 (튜닝 포인트)
threshold = smoothed.max() * 0.3
min_distance = len(profile) // 12

# valley = -smoothed의 peak
valleys, properties = find_peaks(-smoothed, distance=min_distance,
                                  height=-threshold)

print(f'검출된 valley 수: {len(valleys)}')
print(f'valley 위치: {valleys.tolist()}')
print(f'(글자가 7개면 valley 6개, 8개면 valley 7개여야 정상)')

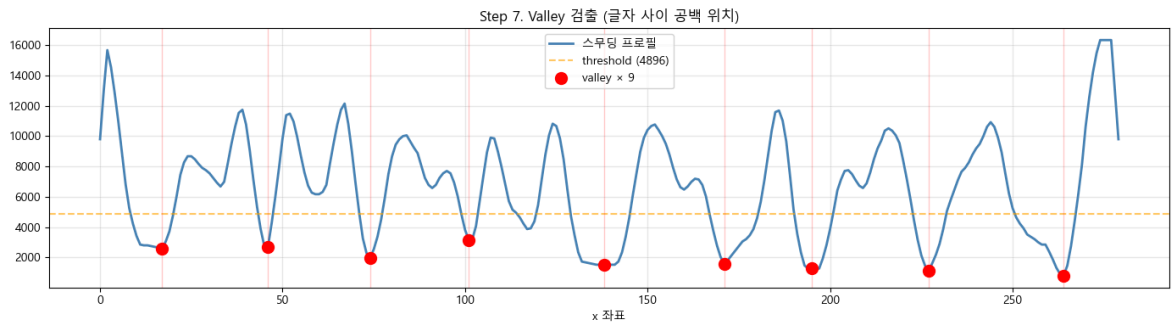
plt.figure(figsize=(14, 4))
plt.plot(smoothed, color='steelblue', linewidth=2, label='스무딩 프로필')
plt.axhline(threshold, color='orange', linestyle='--', alpha=0.6,
             label=f'threshold ({threshold:.0f})')
plt.scatter(valleys, smoothed[valleys], color='red', s=100, zorder=5,
            label=f'valley × {len(valleys)}')
for v in valleys:
    plt.axvline(v, color='red', alpha=0.2, linewidth=1)
plt.title('Step 7. Valley 검출 (글자 사이 공백 위치)')
plt.xlabel('x 좌표')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# 진단
expected_valleys = (EXPECTED_CHARS or 7) - 1
if len(valleys) == expected_valleys:
    print(f'✓ 예상 valley 수와 일치 ({expected_valleys}개)')
elif len(valleys) < expected_valleys:
    print(f'⚠ valley 부족 ({len(valleys)} < {expected_valleys}) - 글자가 붙어 있음')
    print('  해결: threshold를 0.5~0.7로 높이거나 distance를 줄여보세요')
else:
    print(f'⚠ valley 과다 ({len(valleys)} > {expected_valleys}) - 한 글자가 둘로 나뉘어 있음')
    print('  해결: threshold를 0.1~0.2로 낮추거나 distance를 늘려보세요')
```

검출된 valley 수: 9

valley 위치: [17, 46, 74, 101, 138, 171, 195, 227, 264]

(글자가 7개면 valley 6개, 8개면 valley 7개여야 정상)



⚠ valley 과다 (9 > 6) - 한 글자가 둘로 쪼개졌을 가능성
 해결: threshold를 0.1~0.2로 낮추거나 distance를 늘려보세요

Step 8. 글자 경계 결정

valley 위치에 시작(0)과 끝(width)을 추가하여 최종 글자 경계를 만듭니다.

```
boundaries = [0, valley1, valley2, ..., width]
```

인접한 두 boundary 사이가 한 글자 영역이 됩니다.

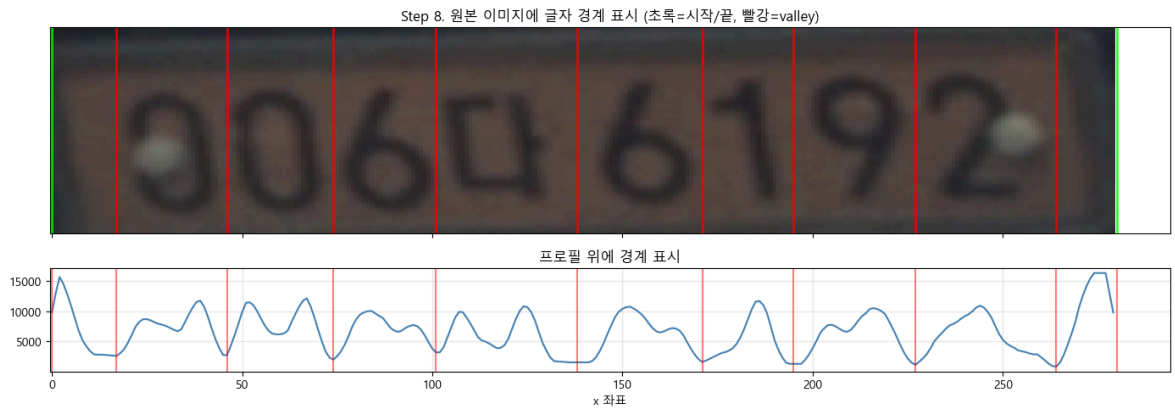
```
In [9]: boundaries = [0] + list(valleys) + [len(profile)]
print(f'경계 수: {len(boundaries)} (글자 영역: {len(boundaries) - 1}개)')
print(f'경계 좌표: {boundaries}')

# 시각화: 원본 이미지 위에 경계선 표시
fig, axes = plt.subplots(2, 1, figsize=(14, 5),
                        gridspec_kw={'height_ratios': [2, 1]}, sharex=True)
axes[0].imshow(cv2.cvtColor(rectified_plate, cv2.COLOR_BGR2RGB), aspect='auto')
for i, b in enumerate(boundaries):
    color = 'lime' if i in (0, len(boundaries)-1) else 'red'
    axes[0].axvline(b, color=color, linewidth=2, alpha=0.8)
axes[0].set_title('Step 8. 원본 이미지에 글자 경계 표시 (초록=시작/끝, 빨강=valley)')
axes[0].set_yticks([])

axes[1].plot(smoothed, color='steelblue', linewidth=1.5)
for b in boundaries:
    axes[1].axvline(b, color='red', alpha=0.5, linewidth=1.5)
axes[1].set_title('프로필 위에 경계 표시')
axes[1].set_xlabel('x 좌표')
axes[1].grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

경계 수: 11 (글자 영역: 10개)

경계 좌표: [0, np.int64(17), np.int64(46), np.int64(74), np.int64(101), np.int64(138), np.int64(171), np.int64(195), np.int64(227), np.int64(264), 280]



Step 9. 글자별 크롭

각 경계 사이 영역을 잘라 글자별 이미지를 만듭니다. 폭이 5픽셀 미만인 영역은 노이즈로 간주해 제외합니다.

```
In [10]: char_crops = []
for i in range(len(boundaries) - 1):
    x1, x2 = boundaries[i], boundaries[i + 1]
    width = x2 - x1
    if width < 5:
        print(f' 스킵: 글자 {i} (폭 {width} < 5)')
        continue
    char_crops.append({
        'image': rectified_plate[:, x1:x2],
        'x_range': (x1, x2),
        'width': width,
    })

print(f'\n최종 분리된 글자 수: {len(char_crops)}')
for i, c in enumerate(char_crops):
    print(f' 글자 {i}: x={c["x_range"]}, width={c["width"]}')

# 시각화: 글자별 크롭
n = len(char_crops)
if n > 0:
    fig, axes = plt.subplots(1, n, figsize=(2 * n, 2.5))
    if n == 1:
        axes = [axes]
    for i, (ax, crop) in enumerate(zip(axes, char_crops)):
        ax.imshow(cv2.cvtColor(crop['image'], cv2.COLOR_BGR2RGB))
        ax.set_title(f'#{i}\nw={crop["width"]}', fontsize=10)
        ax.axis('off')
    plt.suptitle('Step 9. 분리된 개별 글자', y=1.02)
    plt.tight_layout()
    plt.show()
```

최종 분리된 글자 수: 10

글자 0: x=(0, np.int64(17)), width=17
글자 1: x=(np.int64(17), np.int64(46)), width=29
글자 2: x=(np.int64(46), np.int64(74)), width=28
글자 3: x=(np.int64(74), np.int64(101)), width=27
글자 4: x=(np.int64(101), np.int64(138)), width=37
글자 5: x=(np.int64(138), np.int64(171)), width=33
글자 6: x=(np.int64(171), np.int64(195)), width=24
글자 7: x=(np.int64(195), np.int64(227)), width=32
글자 8: x=(np.int64(227), np.int64(264)), width=37
글자 9: x=(np.int64(264), 280), width=16

Step 9. 분리된 개별 글자



Step 10. 글자 수 보정 (Refinement)

분리된 글자 수가 기대치(7 또는 8)와 다를 때 자동 보정합니다.

- **너무 많으면:** 가장 좁은 영역을 인접 영역에 병합 (작은 점·노이즈로 간주)
- **너무 적으면:** 가장 넓은 영역을 절반으로 분할 (글자가 붙어있던 경우로 간주)

이 보정은 휴리스틱이므로 완벽하지 않습니다 — 실패 케이스의 단서가 됩니다.

```
In [14]: def _refine_crops(crops, target_count, source_image=None):
          """원본 _refine_crops에 image 재크롭 보강."""
          crops = sorted(crops, key=lambda c: c['x_range'][0])

          while len(crops) > target_count:
              narrowest_idx = min(range(len(crops)), key=lambda i: crops[i]['width'])
              if narrowest_idx == 0:
                  merge_target = 1
              elif narrowest_idx == len(crops) - 1:
                  merge_target = len(crops) - 2
              else:
                  left_w = crops[narrowest_idx - 1]['width']
                  right_w = crops[narrowest_idx + 1]['width']
                  merge_target = narrowest_idx - 1 if left_w < right_w else narrowest_idx + 1
              a, b = sorted([narrowest_idx, merge_target])
              new_x_range = (crops[a]['x_range'][0], crops[b]['x_range'][1])
              crops[a]['x_range'] = new_x_range
              crops[a]['width'] = new_x_range[1] - new_x_range[0]
              if source_image is not None:
                  crops[a]['image'] = source_image[:, new_x_range[0]:new_x_range[1]]
              del crops[b]

          while len(crops) < target_count:
              widest_idx = max(range(len(crops)), key=lambda i: crops[i]['width'])
              old = crops[widest_idx]
              mid = (old['x_range'][0] + old['x_range'][1]) // 2
              new_left = {'x_range': (old['x_range'][0], mid),
                          'width': mid - old['x_range'][0]}
              new_right = {'x_range': (mid, old['x_range'][1]),
                          'width': old['x_range'][1] - mid}
              if source_image is not None:
```

```

        new_left['image'] = source_image[:, new_left['x_range'][0]:new_left[
        new_right['image'] = source_image[:, new_right['x_range'][0]:new_rig
        crops[widest_idx] = new_left
        crops.insert(widest_idx + 1, new_right)

    return crops

before_count = len(char_crops)
target = EXPECTED_CHARS or (7 if 6 <= before_count <= 7 else 8)

if before_count != target:
    print(f'보정 시작: {before_count}개 → {target}개')
    char_crops = _refine_crops(char_crops, target, source_image=rectified_plate)
    print(f'보정 완료: {len(char_crops)}개')
else:
    print(f'보정 불필요 (이미 {target}개)')

# 한글 자리 표시
if len(char_crops) == 7:
    char_crops[2]['type'] = 'hangul'
    for i in [0, 1, 3, 4, 5, 6]:
        char_crops[i]['type'] = 'digit'
elif len(char_crops) == 8:
    char_crops[3]['type'] = 'hangul'
    for i in [0, 1, 2, 4, 5, 6, 7]:
        char_crops[i]['type'] = 'digit'

# 최종 시각화
n = len(char_crops)
fig, axes = plt.subplots(1, n, figsize=(2 * n, 2.5))
if n == 1:
    axes = [axes]
for i, (ax, crop) in enumerate(zip(axes, char_crops)):
    ax.imshow(cv2.cvtColor(crop['image'], cv2.COLOR_BGR2RGB))
    ctype = crop.get('type', '?')
    color = 'green' if ctype == 'hangul' else 'blue'
    ax.set_title(f'#{i} [{ctype}]\nw={crop["width"]}', fontsize=10, color=color)
    ax.axis('off')
    for spine in ax.spines.values():
        spine.set_visible(False)
plt.suptitle('Step 10. 최종 분리 결과 (한글 자리 = 초록)', y=1.02)
plt.tight_layout()
plt.show()

```

보정 시작: 10개 → 7개

보정 완료: 7개

Step 10. 최종 분리 결과 (한글 자리 = 초록)



검증: 원본 함수 호출 결과와 비교

수동 단계와 동일한 결과가 나오는지 원본 `split_by_projection` 을 호출해 비교합니다.

```
In [15]: # 원본 함수 정의 (디버깅용으로 동일 코드 복사)
def split_by_projection(rectified_plate, expected_chars=None):
    gray = cv2.cvtColor(rectified_plate, cv2.COLOR_BGR2GRAY)
    if len(rectified_plate) > 0:
        binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_C
        h = binary.shape[0]
        binary = binary[int(h*0.1):int(h*0.9), :]
        profile = np.sum(binary, axis=0)
        kernel_size = max(3, len(profile) // 50)
        smoothed = np.convolve(profile, np.ones(kernel_size) / kernel_size, mode='sa
        threshold = smoothed.max() * 0.3
        valleys, _ = find_peaks(-smoothed, distance=len(profile) // 12, height=-thre
        boundaries = [0] + list(valleys) + [len(profile)]
        char_crops = []
        for i in range(len(boundaries) - 1):
            x1, x2 = boundaries[i], boundaries[i + 1]
            if x2 - x1 < 5:
                continue
            char_crops.append({
                'image': rectified_plate[:, x1:x2],
                'x_range': (x1, x2),
                'width': x2 - x1,
            })
        if expected_chars and len(char_crops) != expected_chars:
            char_crops = _refine_crops(char_crops, expected_chars,
                                       source_image=rectified_plate)

        if len(char_crops) == 7:
            char_crops[2]['type'] = 'hangul'
            for i in [0, 1, 3, 4, 5, 6]:
                char_crops[i]['type'] = 'digit'
        elif len(char_crops) == 8:
            char_crops[3]['type'] = 'hangul'
            for i in [0, 1, 2, 4, 5, 6, 7]:
                char_crops[i]['type'] = 'digit'
        return char_crops

# 원본 함수 호출
result = split_by_projection(rectified_plate, expected_chars=EXPECTED_CHARS)

print(f'원본 함수 결과: {len(result)}개 글자')
for i, c in enumerate(result):
    ctype = c.get('type', '?')
    print(f'  #{i} [{ctype}] x={c["x_range"]}, width={c["width"]}')

# 시각화
n = len(result)
fig, axes = plt.subplots(1, n, figsize=(2 * n, 2.5))
if n == 1:
    axes = [axes]
for i, (ax, crop) in enumerate(zip(axes, result)):
    ax.imshow(cv2.cvtColor(crop['image'], cv2.COLOR_BGR2RGB))
    ctype = crop.get('type', '?')
    color = 'green' if ctype == 'hangul' else 'blue'
    ax.set_title(f'#{i} [{ctype}]', fontsize=10, color=color)
    ax.axis('off')
plt.suptitle(f'원본 함수 결과 (expected_chars={EXPECTED_CHARS})', y=1.02)
```

```
plt.tight_layout()
plt.show()
```

원본 함수 결과: 7개 글자

```
#0 [digit] x=(0, np.int64(46)), width=46
#1 [digit] x=(np.int64(46), np.int64(74)), width=28
#2 [hangul] x=(np.int64(74), np.int64(101)), width=27
#3 [digit] x=(np.int64(101), np.int64(138)), width=37
#4 [digit] x=(np.int64(138), np.int64(171)), width=33
#5 [digit] x=(np.int64(171), np.int64(227)), width=56
#6 [digit] x=(np.int64(227), 280), width=53
```

원본 함수 결과 (expected_chars=7)



파라미터 튜닝 — Threshold 비교

`threshold = smoothed.max() * ratio` 의 비율을 바꾸며 valley 검출이 어떻게 달라지는지 비교합니다. 실패 케이스에서 적절한 비율을 찾는 데 유용합니다.

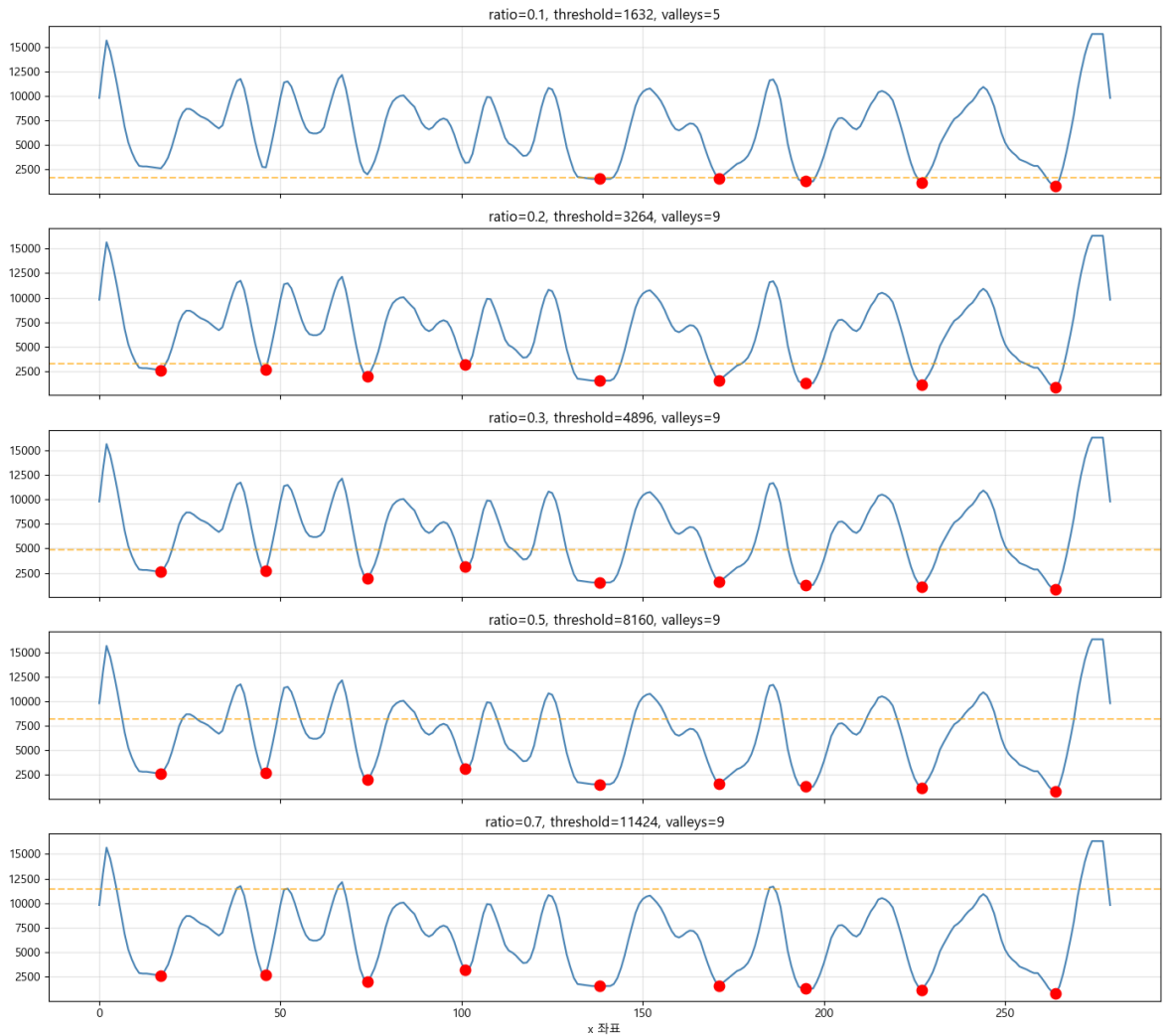
```
In [16]: # 여러 threshold ratio로 비교
ratios = [0.1, 0.2, 0.3, 0.5, 0.7]
fig, axes = plt.subplots(len(ratios), 1, figsize=(14, 2.5 * len(ratios)), sharex)

for ax, ratio in zip(axes, ratios):
    th = smoothed.max() * ratio
    vs, _ = find_peaks(-smoothed, distance=len(profile) // 12, height=-th)

    ax.plot(smoothed, color='steelblue', linewidth=1.5)
    ax.axhline(th, color='orange', linestyle='--', alpha=0.6)
    ax.scatter(vs, smoothed[vs], color='red', s=80, zorder=5)
    ax.set_title(f'ratio={ratio}, threshold={th:.0f}, valleys={len(vs)}')
    ax.grid(alpha=0.3)

axes[-1].set_xlabel('x 좌표')
plt.tight_layout()
plt.show()

print('valley 수 = 글자 수 - 1 이 되는 ratio가 최적입니다.')
```



valley 수 = 글자 수 - 1 이 되는 ratio가 최적입니다.

파라미터 튜닝 — Distance 비교

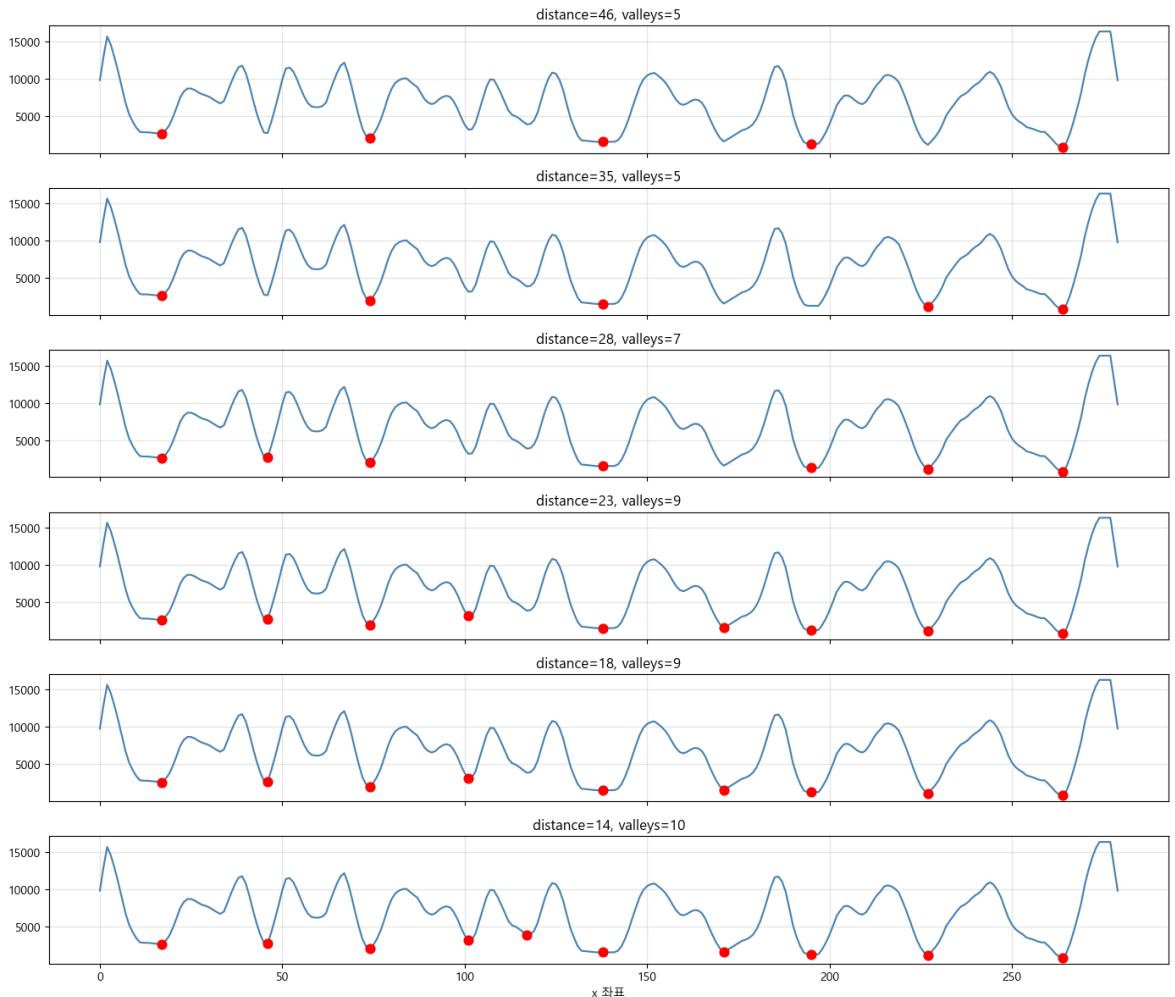
distance 파라미터는 valley 사이의 최소 간격입니다. 글자 폭이 비슷하다고 가정할 때 $\text{len}(\text{profile}) // n$ ($n = \text{글자 수} + 1$) 정도가 적절합니다.

```
In [17]: distances = [len(profile) // 6, len(profile) // 8, len(profile) // 10,
                      len(profile) // 12, len(profile) // 15, len(profile) // 20]

fig, axes = plt.subplots(len(distances), 1, figsize=(14, 2 * len(distances)),
                          sharex=True)

for ax, dist in zip(axes, distances):
    vs, _ = find_peaks(-smoothed, distance=dist, height=-th)
    ax.plot(smoothed, color='steelblue', linewidth=1.5)
    ax.scatter(vs, smoothed[vs], color='red', s=60, zorder=5)
    ax.set_title(f'distance={dist}, valleys={len(vs)}')
    ax.grid(alpha=0.3)

axes[-1].set_xlabel('x 좌표')
plt.tight_layout()
plt.show()
```



종합 진단 함수

여러 이미지를 한 번에 처리하며 어디서 분리가 실패하는지 빠르게 파악할 수 있는 함수입니다.

```
In [18]: def diagnose(image_path, expected_chars=7, save_path=None):
    """한 장의 이미지에 대한 종합 진단 - 모든 단계를 1장의 figure로."""
    img = load_image_safe(image_path)
    if img is None:
        print(f'이미지 로드 실패: {image_path}')
        return

    # 처리 단계
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) if len(img.shape) == 3 else img
    _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_C
    h = binary.shape[0]
    trimmed = binary[int(h*0.1):int(h*0.9), :]
    profile = np.sum(trimmed, axis=0)
    ks = max(3, len(profile) // 50)
    smoothed = np.convolve(profile, np.ones(ks) / ks, mode='same')
    th = smoothed.max() * 0.3
    valleys, _ = find_peaks(-smoothed, distance=len(profile) // 12, height=-th)
    boundaries = [0] + list(valleys) + [len(profile)]

    # 결과
    crops = split_by_projection(img, expected_chars=expected_chars)
```

```

# 한 번에 그리기
fig = plt.figure(figsize=(14, 8))
gs = fig.add_gridspec(4, max(len(crops), 8), height_ratios=[1, 1, 2, 1.5])

ax_orig = fig.add_subplot(gs[0, :])
ax_orig.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB), aspect='auto')
for b in boundaries:
    ax_orig.axvline(b, color='red', linewidth=1.5, alpha=0.7)
ax_orig.set_title(f'원본 + 글자 경계 ({Path(image_path).name})')
ax_orig.axis('off')

ax_bin = fig.add_subplot(gs[1, :])
ax_bin.imshow(trimmed, cmap='gray', aspect='auto')
ax_bin.set_title('이진화 (트리밍 후)')
ax_bin.axis('off')

ax_prof = fig.add_subplot(gs[2, :])
ax_prof.plot(smoothed, color='steelblue', linewidth=1.5)
ax_prof.scatter(valleys, smoothed[valleys], color='red', s=80, zorder=5)
ax_prof.axhline(th, color='orange', linestyle='--', alpha=0.6)
ax_prof.set_title(f'프로필 (valleys={len(valleys)}, expected={expected_chars})')
ax_prof.grid(alpha=0.3)

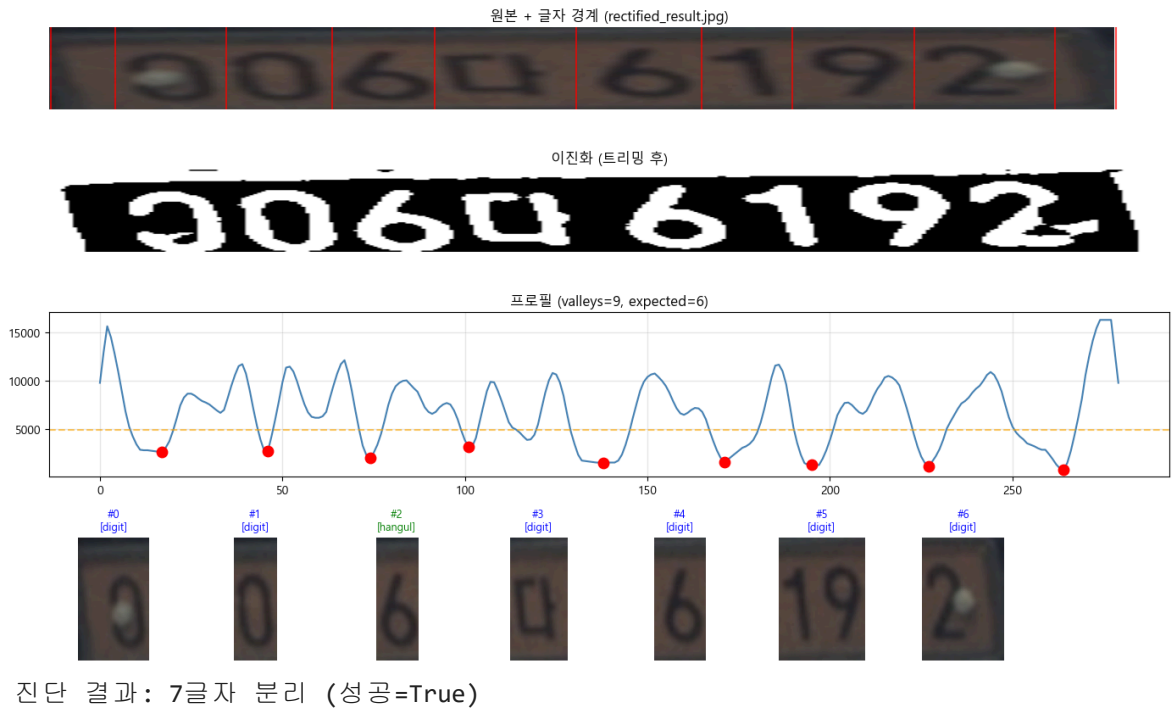
n = len(crops)
for i in range(min(n, 8)):
    ax = fig.add_subplot(gs[3, i])
    ax.imshow(cv2.cvtColor(crops[i]['image'], cv2.COLOR_BGR2RGB))
    ctype = crops[i].get('type', '?')
    color = 'green' if ctype == 'hangul' else 'blue'
    ax.set_title(f'#{i}\n[{ctype}]', fontsize=9, color=color)
    ax.axis('off')

plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=120, bbox_inches='tight')
plt.show()

return {
    'n_chars': len(crops),
    'expected': expected_chars,
    'success': len(crops) == expected_chars,
    'valleys': len(valleys),
    'crops': crops,
}

# 사용 예
result = diagnose(PLATE_IMAGE_PATH, expected_chars=EXPECTED_CHARS)
print(f'\n진단 결과: {result["n_chars"]}글자 분리 (성공={result["success"]})')

```

배치 진단 (여러 이미지 한 번에)

폴더 안의 모든 번호판 이미지에 대해 분리 결과를 일괄 출력하여 어떤 케이스에서 실패하는지 패턴을 파악할 수 있습니다.

```
In [20]: def batch_diagnose(image_dir, expected_chars=7, max_images=10):
    """폴더 내 이미지들을 일괄 처리하며 성공/실패 통계 출력."""
    image_dir = Path(image_dir)
    if not image_dir.exists():
        print(f'폴더 없음: {image_dir}')
        return

    images = []
    for ext in ('*.jpg', '*.jpeg', '*.bmp', '*.png'):
        images.extend(image_dir.glob(ext))
        images.extend(image_dir.glob(ext.upper()))
    images = images[:max_images]

    if not images:
        print(f'이미지 없음: {image_dir}')
        return

    success = 0
    fails = []

    for img_path in images:
        img = load_image_safe(img_path)
        if img is None:
            continue
        crops = split_by_projection(img, expected_chars=expected_chars)
        if len(crops) == expected_chars:
            success += 1
        else:
            fails.append((img_path.name, len(crops)))

    print(f'성공: {success} / {len(images)} ({success/len(images)*100:.1f}%)')
```

```
print(f'\n실패 케이스:')
for name, n in fails:
    print(f' {name}: {n}글자 분리됨')

batch_diagnose('./test_plates', expected_chars=7)
#print('# 실제 폴더 경로를 지정해 호출하세요:')
#print('# batch_diagnose("./test_plates", expected_chars=7)')
```

이미지 없음: test_plates

정리 — 흔한 실패 패턴과 해결책

증상	원인	해결
valley가 너무 적음	글자가 붙어 있거나 임계값이 낮음	<code>threshold * 0.5~0.7</code> 로 상향
valley가 너무 많음	한글이 자모로 분리됨	<code>distance</code> 를 늘리거나 <code>threshold * 0.1~0.2</code> 로 하향
한글 자리가 매우 좁음	한글 글자 폭이 숫자보다 좁게 잡힘	한글 자리에 좌우 padding 추가
첫.마지막 글자가 잘림	번호판 정렬이 부정확	rectification 단계 점검
결과가 매번 들쭉날쭉	조명/오염 변동	CLAHE 같은 전처리 강화

다음 단계

이 노트북에서 분리가 잘 되는 케이스의 파라미터를 찾고, 그 파라미터를 `02_anpr_pipeline.py` 의 ANPR 파이프라인에 반영하세요. 또한 실패 케이스의 특징을 모아두면 다음 데이터 수집·라벨링 시 우선순위를 정할 수 있습니다.